

AI活用学習ノート

AI使ってプログラミング

2025年11月13日

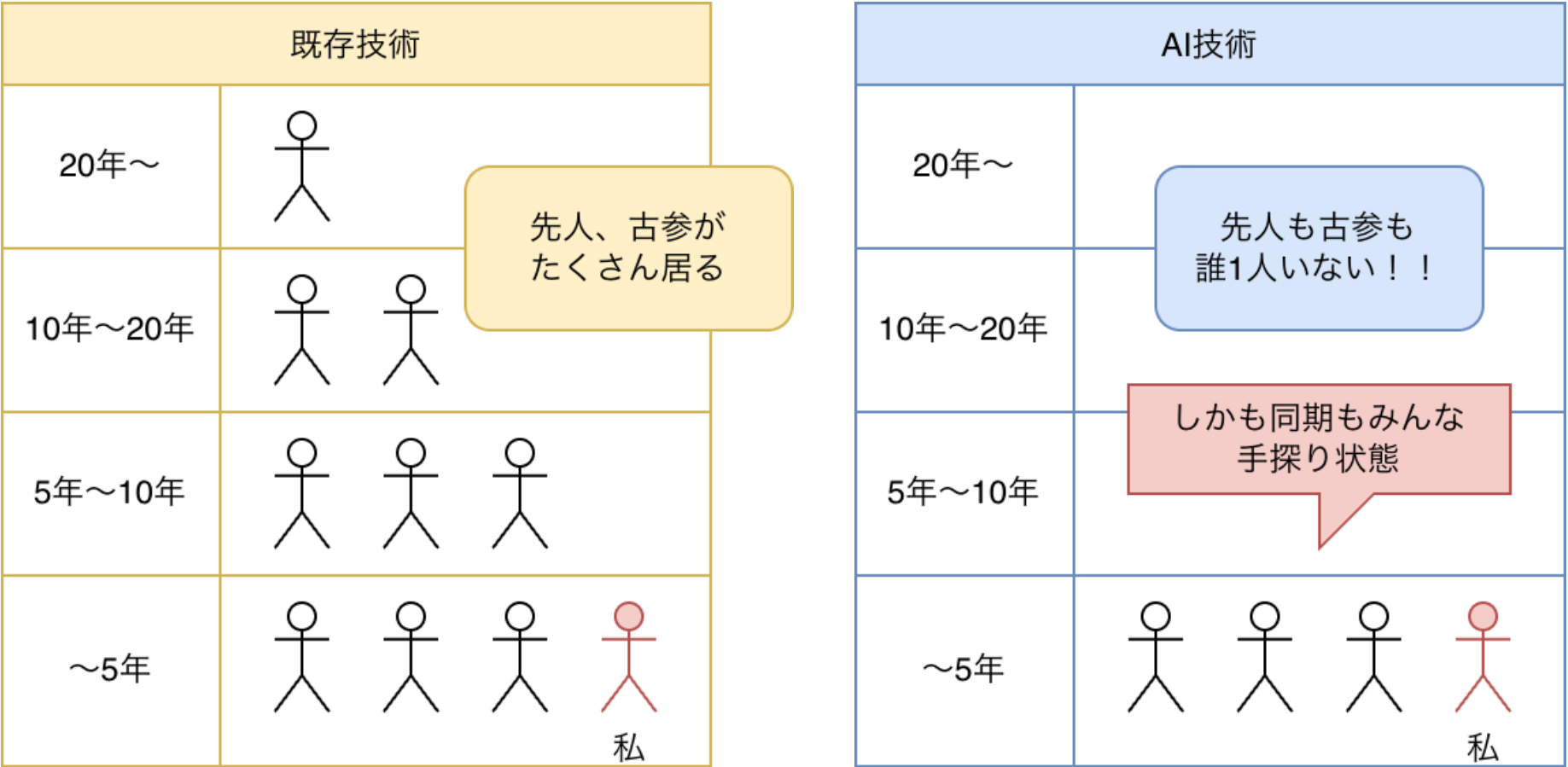
© 2025 nab@garegga.net

§ 1. 導入：なぜ今AIを推すのか？

答え：自己投資先として良コスパだから

i このページは筆者の主観です
まとめると「世界中みんな初心者だからコスパよくね？」です

1. 生成AI活用に古参はいない



2. 生成AI活用は黎明期

生成AIの活用は現在、黎明期です。要は、**全員**まとめて初心者で、**活用法も業務もこれから確立していく時期**という事です。

同じ時間をかけて勉強したとしても、古参がたくさんいる既存技術より、**誰もいないAI技術の方が今後の活躍の可能性が高い**です。

3. ただし、リスクもあり

黎明期技術のよくあるやつです。

- 習得方法がない(要試行錯誤)
- 評価基準もない(給料上がらない)
- 技術が無価値に収束(努力が無駄になる)

i 今後「生成AI」を単に「AI」と略します

本レクチャーでの学び

1. 取り上げる手法

本レクではバイブコーディング(もどき)を取り上げます

バイブコーディング(vibe coding)とは「**日常会話ベースでAIに開発をまるっと任せる手法**」です。「もどき」をつけた理由は筆者自身も初心者だからです。

2. 段取り

下記の順番で解説していきます。

1. ツールの導入
2. 指示1つでWEBサイトの表紙(センス高め)を作成
3. もう1つの指示でログイン機能を追加
4. 適当に仕様書いて渡したら
AIが回答するヘルプデスク(ぽいもの)が完成
5. これまで得られた知見の共有
6. 次レクチャーの要望ヒアリング(けっこう切実)

3. レクの内容を身につけるには？

本レクは短時間に凝縮しているので、まずは今からの解説を一通り聞いてみてください。

レク後に本スライドを共有しますので、**書かれている手順を実際に自分で実行**してみてください。

ただし、AIは黎明期です。各手順、特にプロンプトの書き方などには**正解はありません**。当然得られる結果も期待と異なる場合があります。

例えば、不具合などが発生して先に進めなくなったなら、それもAIに修正を依頼して下さい。どんな不具合でどんな風に修正を依頼すればよいか、そういった、**確立した対策はありません。試行錯誤です。しかし、それ故に、その経験値は直接アドバンテージとなります**

i まとめると、自分で手を動かして実践しよう、です

§ 2. ツールの導入

今回はClaude Codeを使います

Claude Codeとは？

Claude Codeは、AIにプログラミングを自然言語で依頼できる、**まさにVibe Coding用のツール**です。自然言語で指示すると、コードの生成・修正・解説を自動で行ってくれます。アプリ形態はコマンドラインです(黒背景のやつ)。あと**有料**です

画面イメージは下記。

```
% claude 2025/09/28 13:49:57

Do you trust the files in this folder?

/Users/nab391/Library/CloudStorage/Dropbox/devel/0252-lab-vibecoding実験/250928-01-capture

Claude Code may read, write, or execute files contained in this directory. This can pose security risks, so only use files from trusted sources.

Learn more ( https://docs.claude.com/s/claude-code-security )

> 1. Yes, proceed
   2. No, exit

Enter to confirm · Esc to exit
```

他選択肢1:codex-cli

もし、**ChatGPTの有料プランを使っているのであれば、codex-cli**という選択肢があります。

こちらは、有料版ChatGPTのユーザーであれば追加料金なしで利用可能です。機能的にはClaude Codeと同等です。[公式サイトはこちら](#)。

他選択肢2:GitHub Copilot CLI

他には、**GitHub Copilot CLI**もあります。

こちらは、GitHub Copilotの有償ユーザーであれば追加料金なしで利用できます。

Claude Codeをインストールする

1. Claude Codeインストール手順

Claude Codeのインストールは右上にもリンクを貼りましたが、[Claude Code公式サイト](#)の導入手順を参照して下さい。実施するのは「インストールと認証」の箇所のみでOKです

完了したら次ページのスライドへ進んでください。

で、初手の `npm` で躓いた方は以下を参照して下さい

2. 参考: そもそも `npm` って何者？

`npm` はNode.jsに付属するコマンドです。そのNode.jsが何かというとJavaScriptの実行環境です。WEBブラウザ上でもJavaScriptは実行できますが、多々の制限があります。Node.jsではそういった制限がありません。Claude CodeはこのNode.jsをベースとしています。

3. MacとLinuxの方

MacとLinuxの方は [Node.js公式のダウンロードサイト](#) からダウンロード、インストールして下さい。そのあと、Claude Codeをインストールして下さい。

4. Windowsの方

Windowsでは、Node.js公式から普通にダウンロード、インストールしても Claude Code用の環境にはなりません。

下記のMicrosoft公式の手順にしたがって、WSL上にNode.js環境を構築して下さい。

[Windows Subsystem for Linux \(WSL2\) に Node.js をインストールする](#)

そのあと、Claude Codeをインストールして下さい。

§ 3. AIでヘルプデスクを作る(PoC)

まずトツプページを作る

1. `claude` を起動

さっそく開発していきます。

まずどこかに空のディレクトリを作成し、そのディレクトリ内で `claude` を起動します。

```
mkdir claude-lec
cd claude-lec
claude
```

起動画面が表示されたらEnterキーを押します。

```
% claude 2025/09/28 13:49:57

Do you trust the files in this folder?

/Users/nab391/Library/CloudStorage/Dropbox/devel/0252-lab-vibecoding実験/250928-01-capture

Claude Code may read, write, or execute files contained in this directory. This can pose security risks, so only use files from trusted sources.

Learn more ( https://docs.claude.com/s/claude-code-security )

> 1. Yes, proceed
   2. No, exit

Enter to confirm · Esc to exit
```

2. プロンプトを入力する

コマンド入力枠が表示されます。

```
> Try "create a util logging.py that..."

? for shortcuts
```

下記のプロンプトを入力します。プロンプトとはAIへの指示です。

社内ヘルプデスクサイトのランディングページを作成してください。モダンなUIで動きのあるページにしてください

今回の生成物は下記3ファイルでした。(AIの応答結果は都度都度変わります) `index.html` を開きます

- `index.html`
- `style.css`
- `script.js`

生成物を確認

サンプル表示枠 `<button type="button" id="btnA" class="btn-iframe" data-target="#frameA" data-src="/sample/02/index.html"></button>`

```
<iframe id="frameA" class="cv-011-01" src="about:blank" loading="lazy" width="1100" height="800" style="border:1;"> </iframe>
```

コメント

プロンプト1つで生成されたページがこれです。

個人的には自分で作るよりよっぽどセンスも技術も高いと感じてます。

ボタンがいくつも表示されていますが、ログイン機能の実装を進めます。

ログイン機能を追加する

1. プロンプトで指示する

ログイン機能の追加も、プロンプトから普通に自然言語で指示します。今回使ったプロンプトは下記です。

ログイン機能を追加して。ユーザー名とパスワードはテキストファイルに保存（DBは使わない）。ログイン先画面は具体的なコンテンツは無しでログアウトボタンを配置。

「DBを使わない」という制限は、今回は実験という位置づけのために設けたものです。

2. 生成物(追加分)

追加されたファイルは下記4つです。既存ファイルにも修正が入っているようです。（中身の精査はしてません）

- dashboard.css
- dashboard.html
- dashboard.js
- users.txt

ログイン機能確認→エラー発生

サンプル表示枠

The screenshot shows a web interface for a helpdesk. At the top, there's a navigation bar with 'ヘルプデスク' (Help Desk) on the left and 'サービス' (Service), 'お問い合わせ' (Contact Us), 'FAQ', and a 'ログイン' (Login) button on the right. The main content area has a dark blue background with a grid pattern. On the left, there's a large text block that says '社内IT問題は 安心し' (In-house IT problems are安心し) and '24時間365日対応の社内ヘルプデスクから業務効率化まで、あなたをサポートします。' (From 24-hour 365-day in-house helpdesk to business efficiency, we support you). Below this is a 'チケット作成' (Create Ticket) button. In the center, there's a white login form titled 'ログイン' (Login). The form has fields for 'ユーザー名' (Username) with 'admin' entered, and 'パスワード' (Password) with '.....' entered. Below the password field is a red error message box that says 'ログイン処理でエラーが発生しました ×' (An error occurred during login processing). At the bottom of the form, there's a section for 'デモ用アカウント:' (Demo Account) with two entries: 'ユーザー名: admin / パスワード: admin123' and 'ユーザー名: yamada / パスワード: password123'. On the right side of the form, there are two buttons: 'ネットワーク' (Network) and 'データ復旧' (Data Recovery).

1. 確認結果

生成された、index.htmlをローカルファイルとしてWEBブラウザで開いたところ、ログイン時にエラーが発生しました。

2. 今後の進め方

エラー対応もAIに任せてみます。次ページから実際にどうやったかを説明します。

このエラー周りの解説はエラー修正後に別途コメントいたします。(今回のレクチャーの趣旨からは外れますので雑談扱いです)

ログインエラーの修正(AI任せ)

1. 実行結果

エラー発生時、WEBブラウザのコンソールに出力されたログは下記のとおりです。

```
index.html:1 Access to fetch at 'file:///Users/nab391/Library/CloudStorage/Dropbox/devel/0252-lab-vibecoding%E5%AE%9F%E9%A8%93/250927-01-index/users.txt' from origin 'null' has been blocked by CORS policy: Cross origin requests are only supported for protocol schemes: chrome, chrome-extension, chrome-untrusted, data, http, https, isolated-app.
```

```
users.txt:1
  Failed to load resource: net::ERR_FAILED
script.js:743 Login error: TypeError: Failed to fetch
    at HTMLFormElement.handleLogin (script.js:719:32)
handleLogin    @    script.js:743
```

2. エラー修正もプロンプトで指示

特に検討もせず、そのまま丸投げしてみました。

ログイン時、下記のエラーが発生した。修正して
<以下、先のエラーのコピペ>

3. AIからの回答(抜粋)

? CORSエラーを修正しました。以下の変更を行いました:

<中略>

これでローカルファイルからでもログイン機能が正常に動作するはずです。ブラウザでindex.htmlを開いてログインを試してください。

 修正できたとの事なので再度確認します

ログイン機能確認(再)

サンプル表示枠 `<button type="button" id="btn015" class="btn-iframe" data-target="#frame015" data-src="/sample/04/index.html"></button>`

`<iframe id="frame015" class="cv-015-frame" src="about:blank" loading="lazy" width="1100" height="800" style="border:1;"> </iframe>`

1. コメント

サンプル表示枠にてログイン機能を確認します。注目点は以下。

- ログインが機能している
- UIが統一感を持っている
- ログイン先画面が装飾過多(空って指示したのに)

2. 実際に操作

サンプル表示枠で操作します。

3. 次ステップ

まず、今回のエラーに対する見立てを説明します。そのあと事例検索の機能を追加します。

今回のエラーの位置づけ

1. なぜエラー例を記載したか？

WEBや動画などAIの活用例はたくさん流通していますが、成功例ばかりです。**黎明期の技術に本当に必要なものは「その成功例に行き着くまでの紆余曲折」**という経験値であって、最後のまとめだけでは足りないと考えています(個人的には)。よって、記載しました。

2. なぜエラーが発生したか？

一発で完成品が出なかった事を疑問視する方もいらっしゃるかと思います。(AIはまだ未熟だ、など)。しかし、今回のエラーは環境要因です。コーディング自体には問題ありませんでした。

一般的にはindex.htmlは、直接WEBブラウザで開かずに、サーバー経由で表示します。今回のエラーはサーバーを経由しなかったために発生しています。

つまり、**そもそも環境について検討も指示もしていなかった点が原因**ということです。

3. 技術的なハードルが高くない？

先ほどのエラーは解消しました。しかし、サーバー経由にするという根本的な対策には至っていません。

ここに技術的なハードルの高さを感じる方もいらっしゃると思いますが、**今は指示が自然言語(日本語)であることと、それに対してAIが回答を生成できているという点に注目**してください。

実際に活用する場合にはご自身のスキルのある場面にて活用ください。今回は例題としてWEBアプリを取り上げた、というだけに過ぎません。

事例検索の機能を追加#1(設計)

1. プロンプトで指示する

事例検索の機能追加も自然言語のプロンプトで指示します。ただし、ログイン機能とは異なり、事例検索の仕様は千差万別ですので、今回は具体的な機能を指定しています。また、生成されたコードだけから全体の把握は困難ですので、ファーストステップとして基本設計の作成を指示しています。

下記を追加仕様とする。基本設計を作成して。設計では使用するバックエンドやライブラリを明示。コード自体はまだ作成しないで。

- ログイン後画面に「事例検索」ボタンを追加
- 事例検索ダイアログの入力箇所はテキストボックス1つのみ
- 検索をクリックするとローカルのテキストとPDFをベース情報として回答を出力する
- 回答の生成には外部AI (gemini) を利用するものとする

要はローカルのテキストを情報源(事例集など)として、AIに回答させる、という内容です。

2. 応答内容(抜粋)

? 事例検索機能の基本設計を作成します。

1. 機能概要
- 3.2 外部API
Google Gemini API
4. ファイル構成
- 5.1 事例検索プロセス
9. 実装フェーズ
10. 必要な準備事項
 1. Gemini API キーの取得
 2. 事例PDFファイルの準備

この設計に基づいて実装を進めますか？

全体で200行程度の回答でした。Gemini APIの利用やローカルPDFをベースとした回答生成など指示内容は組み込まれているようなので、このまま進めます。(技術的内容はパスで。段階を設けたという工夫のお披露目です)

📝 今回のプロンプトの工夫の説明

- ・AIが的を外さないようターゲットを具体的に絞り込む
- ・外してないかチェックを入れるために段階を設ける

事例検索の機能を追加#2(実装)

1. プロンプトで指示する

設計を確認したので、実装を指示します。

テキストを基本設計書.mdに保存。
先の設計に基づいて実装を進めて

2. 応答内容(抜粋)

? 事例検索機能の実装が完了しました！
実装完了内容

1. ファイル構成
 2. 主要機能
 3. 使用方法
 4. 技術仕様
 5. セットアップ手順
 1. <https://makersuite.google.com/app/apikey>でAPIキー取得
 2. config.jsでAPIキー設定
 3. data/cases/フォルダにテキストファイル配置
 4. ブラウザでテスト実行
- システムの準備ができました。APIキーを設定すれば、すぐに事例検索機能をテストできます！

3. セットアップを実施

実装の完了報告と併せて、セットアップ手順が提示されました。手順通りにセットアップも実施。次ページで動作確認します。

 繰り返しですが、AIは毎回出力結果が変わります
臨機応変な対応が必要です。

4. 参考: Gemini APIについて

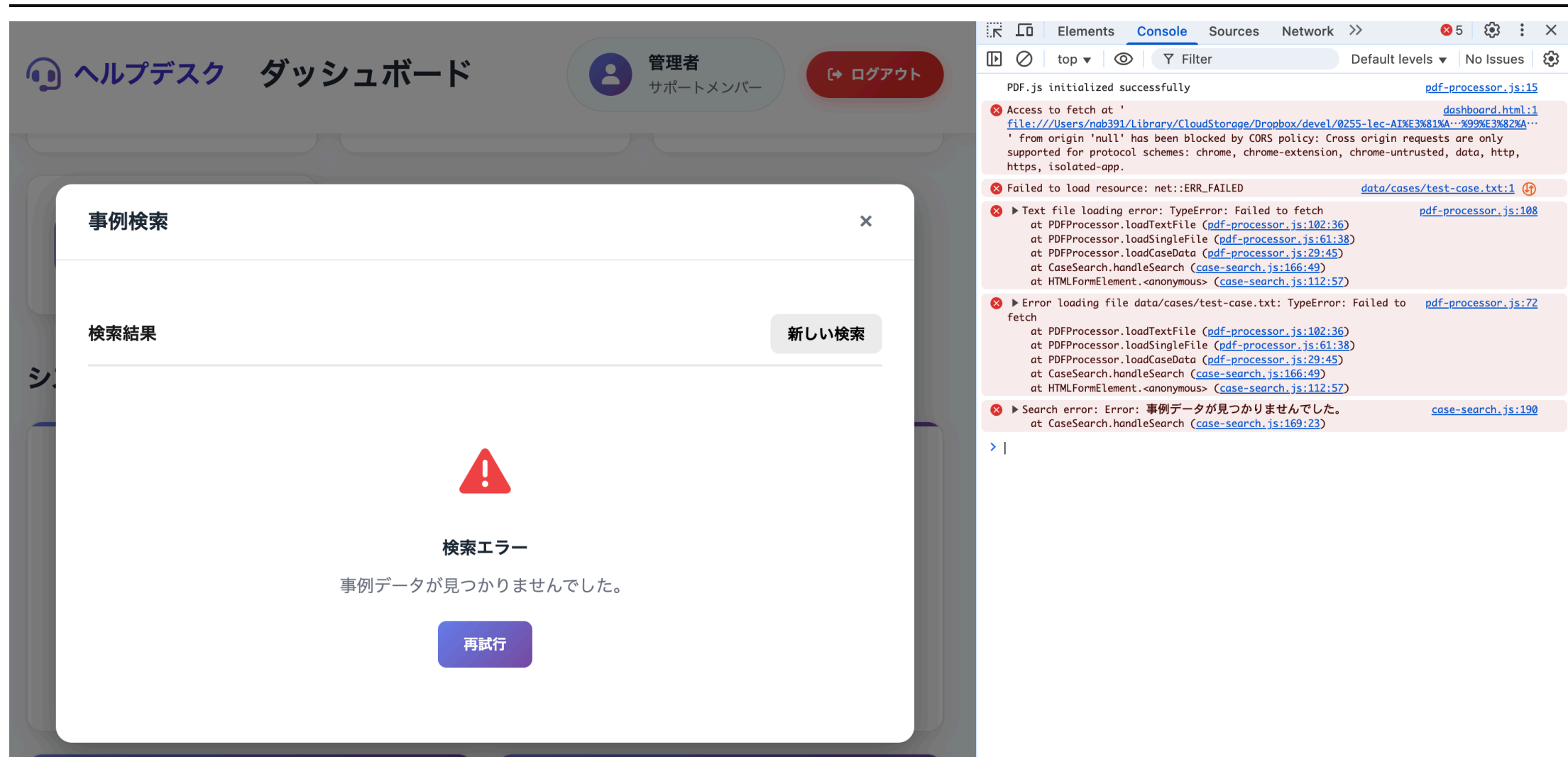
Gemini APIには無料枠があるので、それを利用しています。実験的な実装に役立つと思いますので共有します

- [Gemini API価格表](#)
- [APIキー取得](#)
- [Gemini API quickstart](#)

quickstartのページではプログラミング言語ごとの実装サンプルが用意されています。

事例検索機能確認→エラー発生

サンプル表示枠



1. 注目ポイント

事例検索機能の確認結果をサンプル表示枠に表示します。

質問を投げた時点でエラーが発生する、という結果でした。

2. 実際に操作

~~サンプル表示枠で操作します。~~

3. 次ステップ

先のエラーに続いて、同様にエラー対応を進めます。

エラーの対応(今回もAI任せ)

1. 何回もエラーが発生する件

実際にAIを活用している開発現場においては、1)自動テスト、2)結果判断も自動、3)AIへの不具合修正依頼も自動、の繰り返しを行うように設計しているようです。

今回は**その設計に至る以前の、具体的にどのようなエラーが生じ、それをAI自身がどこまで対策できるのか、といった確認(経験知)**を目的としています。

2. 今回のエラーの内容と指示プロンプト

手短に言うと、サーバー経由じゃないのでローカルの事例データ(テキストやPDF)が読み込めなかった、です。

修正依頼プロンプトは次のとおりです。

下記のエラーが発生し、事例検索ができなかった。原因を調べて対策案を提示して。
調査範囲はコード以外にも実行環境なども含めること。
<以下、エラーメッセージ>

3. AIの出力

AIの応答は下記の通り。根本対策に到達しています。

> 状況整理

<中略：エラーメッセージから状況確認>

原因推定

<中略：「ローカルファイルアクセス制限」と推定>

対策案

<中略：「HTTPサーバー経由で配信する」を提案>

HTTP サーバー配信に切り替えれば、loadCaseData() から先の処理が正常に進み、事例検索 (Gemini問い合わせまで) が実行できるようになります。

対策した結果を次ページにて確認します。

i 実際には他にも不具合がありました

具体的にはgeminiのURL誤り&その周辺です。
取り上げる意義が薄い事と、修正まで何度も指示を繰り返す必要があったため省略しています。

事例検索機能確認(これで完成)

サンプル表示枠 `<button type="button" id="btn21" class="btn-iframe" data-target="#frame21" data-src="/sample/07/index.html"></button>`

`<iframe id="frame21" class="cv-021-frame" src="about:blank" loading="lazy" width="1100" height="800" style="border:1;"> </iframe>`

1. 注目ポイント

サンプル表示枠にて事例検索機能を確認します。注目点は以下。

- 事例検索メニューが追加されている
- 質問入力ダイアログが表示される
- 事例検索が機能している(エラー対策ができています)
- ローカル情報が反映されている
(合言葉: バナナペンギン2025)

2. 実際に操作

サンプル表示枠で操作します。

3. 次ステップ

ここまでの技術的まとめを行います

§ 4. ここまでの技術的まとめ

技術的トピック

AIプログラムで抑えておきたい所をご紹介します

1. 一般のチャット的なAI

ChatGPTなどの一般のチャットAIでは、日本語で質問したら普通に回答が得られます。

このAIへの質問や指示を「プロンプト」と呼びます。

で、AIを使ったプログラムではどうなってるかというと、右

解説は次ページ

2. 今回の生成コード(抜粋)

```
buildPrompt(略) {  
  const systemPrompt = `あなたは社内ヘルプデスクの  
  AIアシスタントです。以下の事例データベースから<略>
```

【回答の形式】

1. 該当する事例があれば事例番号を明記
<略>

【回答のスタイル】

- 丁寧で分かりやすい日本語
<略>`;

```
const userPrompt = `
```

【ユーザーの質問】

```
${query}  
<略>
```

【事例データベース】

```
${caseContent}  
<略>
```

上記の事例データベースを参考に、ユーザーの質問に対する最適な回答を生成してください。<略>`;

プログラム中のプロンプト

1. プロンプトの種類

プロンプトには2種あります。1つはシステムプロンプトで、AIにベースとなる役割を指定するものです。もう1つはユーザープロンプトで、AIに個別に指示・回答するものです。

2. 今回のプログラム内の変数

今回のプログラムだけの個別のお話ですが、

変数として `${query}` は、画面内でユーザーが入力した質問文章そのものを格納しています。

もう1つの変数 `${caseContent}` は、事例データベースから取得したテキスト文章を格納しています。

3. さきほどのコードをどう見るか

```
buildPrompt(略) {  
  const systemPrompt = `  
  ※ここにシステムプロンプト（役割）  
  `;  
  const userPrompt = `  
  ※ここからユーザープロンプト（個別指示）
```

【ユーザーの質問】

`${query}` → ※ユーザーが入力した質問文

【事例データベース】

`${caseContent}` → ※事例から取得したテキスト文章

上記の事例データベースを参考に、ユーザーの質問に対する最適な回答を生成してください。<略>`;

システムプロンプトとして日本語で役割を指示しており、ユーザープロンプトとしてChatGPTに質問するのと同じ感じで、普通に日本語で質問しています。

プログラミングにおいても、AIへの指示は普通に日本語で行っている。という事をお伝えしておきます。

プログラムはプロンプトを流してるだけ？

1. 前ページのお話を要約

プログラムの中からAIを利用するときは、自然言語のテキスト(プロンプト)を用意して、AIに渡してその回答を受け取ってるだけ。です。

2. 逆に言えば

プログラムの仕組み(外枠)だけ用意すれば、中身のプロンプトを入れ替えるだけで別の機能を持たせられるのでは？

3. 半分正解

同じプログラムでもプロンプトだけ入れ替えて別の機能を持たせることは可能です。[サンプル@Qiita](#)

今回もヘルプデスクではなく、書記担当という役割を与えれば議事録作成とか別の機能を実現できます。

4. プロンプトは重要

指示プロンプトを変えるだけで機能がガラッと変わるということは、プロンプトの内容次第で回答の精度も変わるという意味でもあります。

ですので、**ある機能を突き詰めるためにプロンプトを磨き上げる**ことは重要です。次ページでやります。

5. 仕組みも重要

しかしながら、**AI活用を全体的に学ぶ場合にはプロンプトだけでなく、「どういった仕組みを用いているのか」にも注目すべきです。**

仕組みとは、例えばMCPやAIEージェントなどです。

プロンプトを磨く

1.とりあえず基本構成の紹介

プロンプトの(私の)基本的な書き方は下記の通りです。

要素	内容	例
目的	何を達成したいのか(最重要)	「線形代数レクチャー資料の構成案を作りたい」
背景	状況や文脈・ストーリー(超重要)	「対象は数学初心者。用途はAIの仕組み理解」
役割	AIに担わせたい専門性や職種、人格、視点	「あなたは教育設計に長けた数学講師です」
出力形式	ファイルタイプ・粒度・段階	「箇条書きで3段階構成」「Markdownで」
制約条件	限定や禁止事項	「専門用語は最小限」「300字以内」「数式はLaTeXで」
例・書式 ※1	期待する出力例	「例:1.導入、2.理論、3.演習」

目的は当然ですが、背景も非常に重要です。最近はコンテキストエンジニアリングという言葉も出てきました。

2.今回は

プロンプトとしては書き切れなかったため、「仕様書的な物を予め用意しておく」という形で進めてみました。

また、AIの中でバグ修正まで完結させるよう、テストの指示も含めています。仕様書の中身は一応掲載しておきます。(説明はパスします)

```
# PoC開発仕様（AI自動開発用仕様書）
## 1. 目的
- 社内ヘルプデスクサイトのPoCを作成し、AIによる自動開発の有用性を検証する。
- 特に以下2点を目的とする：
  1. トップ画面をモダンで動きのあるUIとする。
  2. 設計 → 実装 → テスト → 修正のサイクルを、AIが自律的に遂行できることを示す。

## 2. 機能一覧
- トップ画面
  - モダンなUI、ログインボタンを配置。モーションやアニメーションを採用。
- ログイン機能
  - ローカルファイルの認証情報を参照してユーザー認証。成功時に事例検索画面へ遷移。
```


仕様書的なテキスト#1(パス)

- 事例検索画面
 - テキストボックス入力から質問を受け付け、Gemini APIを使って回答生成。
- ファイル検索機能
 - 回答生成時にローカルテキスト/PDFを参照。ファイルアクセス失敗時は明示的にエラーを出す。

3. 詳細仕様

3.1 ログイン仕様

- 認証情報は`data/users.txt`に保存。
- フォーマット：
user1,password1
user2,password2
- 認証方法：
 - 入力ユーザー名とパスワードを照合。
 - 一致しなければ「認証エラー」としてエラーメッセージを表示。
 - 暗号化・権限情報・外部認証・DBは使用しない（PoCのため簡易構成）。

3.2 質問・回答仕様

- Gemini APIを呼び出し、ローカルファイル内容をベース情報として回答を生成。
- ベース情報格納先：`knowledge/faq.txt`および`knowledge/faq.pdf`
- Gemini APIの利用手順：
 - <https://ai.google.dev/gemini-api/docs/quickstart#javascript>
- Gemini API キー格納場所：`.env`（GEMINI_API_KEY=...）

- 呼び出し手順：
 1. `faq.txt`、`faq.pdf`から関連情報（テキスト）を読み取り。
 2. 質問内容と読み取ったテキストと一緒にGemini APIへPOST。
 3. 具体的なプロンプトは`data/\*\_prompt.txt`を参照。
 4. 回答テキストを結果欄に表示。
- 問い合わせのプロンプトは下記に分離して格納すること。（検証&調整のため）
 - システムプロンプト：`data/system\_prompt.txt`
 - ユーザープロンプト：`data/user\_prompt.txt`

3.3 実行・参照環境

- フロントエンドはHTML + JS + CSSで構成。
- バックエンドは簡易サーバー（Node.js + Express）を利用。
- `npm run start`でローカル起動し、`http://localhost:3999`で確認可能。

仕様書的なテキスト#2(パス)

4. 開発体制・手順 (AI実行フロー)

1. 設計フェーズ

- 以下3文書を生成する：
 - ``docs/basic_design.md``
 - ``docs/test_plan.md``
 - ``docs/error_handling.md``
- 各文書には以下を含む：
 - ライブラリ・使用技術
 - 実行・テスト環境 (Node, npmコマンド, MCPサーバー)
 - 機能別テストケース
 - 想定エラーと対処法

2. 実装フェーズ

- ``src/``ディレクトリに必要ファイルを生成。
- 生成後、``npm run start``で起動できる状態にする。

3. テストフェーズ

- MCPサーバー「playwright」を使用してE2Eテストを実行。
- 主要テスト項目：
 - WEBページの表示成功／失敗
 - ログイン成功／失敗
 - FAQファイル取得成功／失敗
 - Gemini API 正常応答／URL誤り
- 結果レポートは``reports/test_result.json|md``に保存。

4. 修正フェーズ

- テスト結果にNGがあれば、AIは``docs/error_handling.md``に従って修正。
- 修正後、再テストを実行。
- 同一ループを最大5回まで繰り返す。
- 5回以上繰り返した場合は停止し、現象と推定要因／対策を``reports/cause.md``にまとめる。

5. 成果物一覧

AIによる自動生成物および事前の準備ファイル

- 仕様
 - ``spec/spec.md`` (本仕様書、修正不可)
- 設計
 - ``docs/basic_design.md``、``docs/test_plan.md`` (設計・テスト設計書)
- 実装
 - ``src/`` (コード (HTML、JS、CSS、Node.js))
- データ
 - ``data/users.txt``、``knowledge/faq.txt``、``knowledge/faq.pdf`` (ローカルデータ)
 - 事前にファイルが存在する場合はそれを利用。なければダミーデータを自動生成。
- ログ
 - ``logs/``、``reports/`` (実行ログ、テストレポート)
- 除外
 - ``work/`` (人間の作業用、AIからは修正不可)

6. 成功条件

- 全テスト項目が「成功」と判定される。
- ブラウザ上でトップ画面→ログイン→質問→回答表示が確認できる。
- すべての結果レポートが ``reports/`` に生成される。

7. 想定エラーと検出方法

今回は事前の規定無し

8. 補足

- 環境変数は全て `` .env `` で管理する。

MCPサーバーを導入

1.MCPサーバーとは

端的に述べると「**AIのスキル拡張**」です。

本来、AI、LLMができることは、1)テキスト入力を受け取る。2)入力テキストに対応した返答をテキストで返す。の2点だけです。

入出力に画像や音声を用いたり(マルチモーダル)、必要な情報を外部に取得しに行ったり(RAG)、といった動作はLLMに備わっている機能ではなく、LLMをサポートするように周囲にそれぞれの役割のプログラムを配置しているだけです。

当然、「**もっと様々な機能のプログラムを用意したらもっと色々できるのでは?**」という発想がでてくると思いますが、それを統一規格としてまとめたのがMCPサーバーです。

2.導入理由

今回、MCPサーバーが必要となった理由は、「WEBアプリをAIに自動テストさせる為」です。

自 つまり、「**プロンプトで自動テストを指示**」しても「AI/LLMにはその機能がないので実行してくれない」というお話です。

よって、できる事を増やすために、今回で言うと「**WEBブラウザを通した操作・テストを自動で実行させるため**」に導入という流れです。

具体的には「Playwright MCP」を導入しました。Claude Codeへのインストール手順は下記を参照してください。

[Claude Codeとplaywright mcpを連携させると開発体験が向上するのでみんなやろう](#)

プロンプト & MCPサーバーによる生成物を確認

サンプル表示枠

AIヘルプデスクで、社内問い合わせをスマートに。

モダンなUIとGemini API連携で、よくある質問への回答を高速に提示します。プロトタイプとしてトップ画面・ログイン・FAQ検索・回答表示までを自動化しました。

ログインして試してみる

PoC: トップ画面→ログイン→質問→回答を連続体験できます。

1. 注目ポイント

サンプル表示枠にて生成物を確認します。注目点は以下。

- 一から作り直しの為UIが全く別物
→ UIの制御は別に対策が必要

なお、今回の生成について、エラー確認と対策はAIでほぼ完結しており、一つ一つ人間が対応するという手間はかかっておりません。(一点、Geminiに渡すプロンプトだけは手直ししています)

2. 実際に操作

今回は無し。画像のみです。

3. 次ステップ

AIの実験的な作業はここまでです。

できれば、一度ご自身でも試してみてください。

次章では、この経験を^{てきとうにまとめて}知見へと昇華しておきます。

§ 5. 知見への昇華

思考軸(視点)

ここからはかなり主観と私見メインです。
ご了承ください。~~オチはつけます。~~ だめでした

1.思考軸とは

個人的な呼び方です。**新しい何かを理解する為に、どういった視点でそれを見るか、**というだけのお話です。ただし、理解が深まるにつれ、増えたり減ったりします。つまり仮説的アプローチでもあります。

2.今回の思考軸

今回は、下記4つの軸(観点)で考えていきます。

- 技術観点
- 機能観点(活用法)
- ソクラテ斯的観点(無知の知)
- AI的直感思考の観点

3.技術観点

単純に「**技術的にどういった構成・作りになっているか**」という**視点**です。最悪、プログラムできたらいっかーレベルでもあります。

4.機能観点

技術的観点から一步ユーザー側に立った視点です。「**その技術名称が示す機能はユーザーにどういった活用法を提示できるのか?**」という見方です。

5.ソクラテ斯的観点

ご存じの方も多いと思いますが、**無知(未知)とそれを知る知(既知)のペアリング**です。

- 既知の知(知っていることを知っている)
- 既知の未知(知っていることを知らない)
- 未知の知(知らないことを知っている)
- 未知の未知(知らないことを知らない)

AI的直感思考

1. 人間における直感

人間における直感とは知識と理解と実践の上に構築される、その人のハイパー判断力です。

消防士歴40年の人の「この建物なんかヤバくない？」

そこにはエビデンスも言語化された説明もありません。しかし、ただの感想でも偏見でもありません。

人間的な直感とはこれまでの積み重ねから脳が潜在的に判断した合理的な結論です。

2. AIにおける直感

AIは原理的に、次に紡ぐ言葉を確率的に算出・決定しています。これを個人的に「AI的直感思考」と呼んでます。

AIの回答が理路整然としている場合も多々ありますが、単に元となる学習データが論理的であるために、そういう見た目の回答に収束しているだけです。

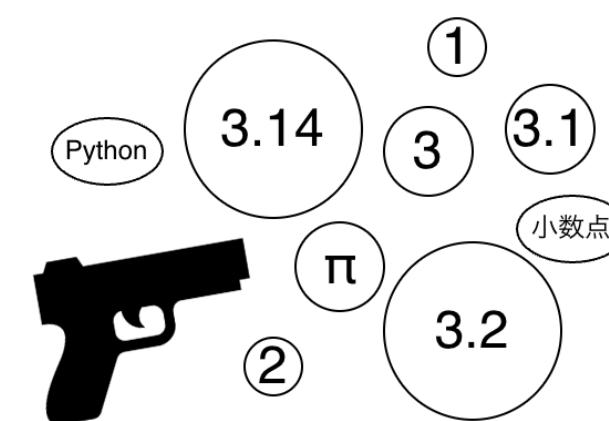
3. AI的直感思考のイメージ

お題「3.14と3.2どちらが大きい？」

4. 論理的思考

- 最初の数字は3で同じ
- 次の数字は1と2で2の方が大きい
- 結論:3.2の方が大きい

5. AI的直感思考



「適当に撃つ」

※予想外の大当たりもあれば大外しもあります。

6. この観点をを用いる理由

AIが「直感ベースの思考をする人間」である、という見方ができる事から、それを前提に、如何にその直感をサポートしていくか、という観点で各手法を組織的・管理的な解釈でまとめられないかと考えたからです。

まとめ方針

1. メイン軸はAI的直感

各技法のまとめにあたって、メイン軸は「AI的直感」とします。理由は「他にその例を見たことがないので、ちょっと挑戦」です。結果、来月には役立たずかもしれませんが、ご了承ください。

2. メイン軸からのアプローチ

まず、AIを「直感ベースの発言者」と見立てます。次に、どのようなサポート/フォロー要素があるかを考えます。最後に、それらサポート要素を技術名と紐付けて解釈します。

とりあえず、AIは超高スキルな新卒コンサル兼エンジニアの位置づけにします。ポテンシャルは超高いけど目の前の現場業務はあまりよくわかってません。

サポート要素は対人間における見識からの抽出です（個人の感想ベースです）。適当に列挙します。

3. 抽出した要素を列挙

メイン軸に沿った要素は次の5つです。具体的な技術要素との紐付けも添えておきます。

・考える方向性を明示する（＝思考の誘導）

→プロンプトエンジニアリング

・エビデンスをとらせる（＝外部情報の取得）

→RAG

・知識の補強をする（＝直感自体の底上げ）

→ファインチューニング

・思考を妨げない（＝雑作業の外部分離）

→MCP

・自律して完結（＝仕組み/組織としてフォロー）

→AIEージェント

では、各要素毎に解説していきます。

考える方向性を明示する(=思考の誘導)

1. AI的直感としてのイメージ

新人AI。スキルはあるけど、現場を知らない。指示したり質問したりしても、行動も回答も要領を得ない。そんな事態に直面したら、改善すべきは指示の内容です。

回答として求めている事や前提を明確にする、アプローチや手段を指定する、周辺事情を共有する等々、状況に応じて様々な要素が思いつきますが、要は、**具体的な回答の形/方向性を明示**しましょう、という事です。

なお、どれだけ細かく指示をしたとしても、AIの回答は直感ベースです。それらを論理的に積み重ねてる訳ではありません。結局はランダムに撃ち抜いて結果を出すだけです。**プロンプトはその撃ち抜く対象や方向の指示**というイメージです。

2. 他の思考軸での解釈

技術観点では、入力テキストが最適化すればするほど、そこから計算される次の単語(回答)が関連性の高い物に絞られます。

機能観点では、プロンプトの書き方の工夫になります。**技法として名前がついており、プロンプトエンジニアリング**と呼ばれます。先のページにて表にまとめてますので、具体的にはそちらを参照ください。

ソクラテス観点では「未知の既知」です。つまり、「知っているのに知らない」状態。噛み砕くと、「学習済みののに、気づいてないし自分から気づくこともない状態」です。

質問の仕方を変えることで「あ、それ知ってるわ」と自分の知識に気づいて回答が引き出せるという感じです。

エビデンスをとらせる(=外部情報の取得)

1. AI的直感としてのイメージ

方向性に合った回答をするようになったけれども、そもそもが直感ベースなので、**具体的な数値や個別情報**が**いい加減(誤ってる)**、という状況とします。

そういった場合、対人間では**エビデンスとして一次情報を確認するように指示する**のが一般的な対応です。

しかし、**AI相手ではそれは無意味**です。なぜなら、指示を受けようがどうしようが、「適当に直感で回答する」以外には何もしない/できないからです。

よって、**回答を監視して、必要に応じて外部情報を取得・フィードバックするサポーターを付ける**といった対策が必要になってきます。

2. 他の思考軸での解釈

技術観点ではLLMを2段階に分けます。「最初の出力をチェックし、必要に応じて外部情報を取得するアプリ」を用意しておき、その情報をフィードバックして2段階目のLLMを実行することで、誤情報を回避した最終回答を出力するという構成です。**この技法はRAG**と呼びます。

機能観点では、RAGは裏方の技術となります。ただし、RAG機能が導入されているかどうかによって、プロンプトの書き方も変わりますので、確認は必要です。

ソクラテス観点では「未知の未知」です。**不正確な情報しか知らないということ**を**自覚できていない状態**です。RAGはそれを精神論ではなく、仕組みとして解決する手法です。AI自体の振る舞いは変わってません。

知識の補強をする(=直感自体の底上げ)

1. AI的直感としてのイメージ

どのような技術や業務においても、勉強する事は求められます。AI側においても同様の事情があります。

AIは汎用的な学習はしていても、個別具体的な事例は未知という状態にあります。よって、それら**事例を追加学習させることで、知識の補強、直感力を底上げしよう**というアプローチです。

イメージ的には、銃で撃ち抜く^ま的/ターゲットをより適切な種類と大きさに取り替えようといった感じです。

やってることは超優秀新卒エンジニアに社内資料を読ませたりする事と同様ですが、**AIは学習はしても記憶しているわけではなく、直感力が鍛えられただけ(内部パラメータが更新されただけ)**という点が異なります。

2. 他の思考軸での解釈

技術観点では、**追加の学習データを用いて、内部のパラメータを更新**する事に該当します。ただし、全パラメータの更新は負荷が高いため、LoRAと呼ばれる、一部パラメータのみを更新する技法が用いられる場合が多いです。これは**ファインチューニング**と呼びます。

機能観点では、ローカルAIなどに事例データを追加学習させる作業になります。ただし、既存資料を適当に読み込ませるのは悪手で、最悪、逆に精度が下がります。よって**学習用資料の準備に非常に負荷がかかります**。

ソクラテス観点では「未知の未知」から「既知の知」に変化する意味合いです。**知識補強という意味では根本解決です。が、パブリックなクラウドAIでは適用できない技法でもあります**。

思考を妨げない(＝雑作業の外部分離)

1. AI的直感としてのイメージ

例えばプログラムコードの解析をする場合、一般的にはIDE(開発用環境)などを用いて、あっちこっちに散らばっている情報を効率的に集約・参照したりします。メモ帳だけだと、優秀なエンジニアでもすぐに限界が来ます。

AIにコード解析を依頼した場合も同様の状況が発生します。素のAIはメモ帳オンリーの状態です。更に、論理的思考ができないAIでは、どれが枝葉でどれが根本であるかの自立的な仕分け能力も備わっていません。**細々した枝葉の調査解析に思考が費やされ、妨げられ、全体的な解析精度は低下**します。

対策は明白で、AIに開発用環境(外部ツール)を提供すれば解決です。**枝葉の作業を外部ツールに任せる仕組みを用意することで、根本の思考を妨げないように**します。

2. 他の思考軸での解釈

これは**MCPという技法**に該当します

技術観点では、RAGの延長です。RAGは外部情報の取得に特化していましたが、「AIの出力をウォッチして、必要なサポートを外部アプリが提供する」というアイデアを踏襲しています。各社が独自に開発していたものを統一規格としたのがMCPです。

機能観点では、AIに様々なスキルを付与する仕組みです。どのようなスキル/機能なのかは制限がなく、一般のプログラムで実現できる事は全てAIに提供できます。

ソクラテス観点は、特に言及ないです。外部のサポートから既知の領域が広がる事には興味はありますが。

自律して完結(=仕組み/組織化)

1. AI的直感としてのイメージ

超スキルが高い従業員がいたとして、都度都度作業を指示するのではなく、まるっと業務を任せたいとします。

具体的には、先のヘルプデスクの開発のように、人間側でテストしたり、バグ修正を指示したりせず、AI内で全部自律的に実行/完結して欲しい、というイメージです。

対人間であれば、業務としてアサインすれば良いだけです。自分の中で役割を分けて作業して貰えればOKです。

しかし、AIは実質、単機能です。特定の役割を指示したら、その役割に応じた対応しかできなくなってしまう。全てを賄う役割を指示しても、狙う的の幅が広がってしまい、逆に精度は落ちます。

対策としては組織化です。管理や作業等の個々の役割に特化した単機能のAIについて、互いにフォローする体制を設ける事で、全体として自律的に動くようにします。

2. 他の思考軸での解釈

技術観点では、RAGやMCPの延長です。ベースとなるLLMの入出力はテキストオンリーで変わっていません。そのLLMを機能的なパーツと捉え、他のサポート的な外部機能(アプリ)をワークフロー的に組み合わせることで、LLM的な思考・判断力と、ソフトウェア的な機能を融合させようというものです。

機能観点では、内部構造を意識する必要はなく、全体として1セットの自立的な機能を提供しています。AIエージェントと呼ばれるものもこの範疇です。

ソクラテス観点では、各AIがそれぞれの役割に特化した結果、(同じモデルであっても)それぞれ既知と未知の異なる領域を担っている事に興味です。(ビジネス観点ではどうでもいいお話ですが)

結局、人間側に求められる能力とは？

本資料をまとめる間に気づいた、雑談です。

1. まずマネジメント能力

ここまで紹介した技法は、端的に述べると、AIに対するマネジメントです。それも、1on1レベルの指示から組織構築までの幅広さです。

ですので、**求められるのは技術的に使える使えないだけでなく、業務&仕組みの観点からの設計・評価能力**ではないかと思われます。特に、個別具体的な業務ドメインに対する理解は必須になると思います。

2. 次に専門性

AIには専門的な役割をアサインする場合があります。ここで開発者に求められるのは「**コンサル/PM的立場で専門家と会話できる能力**」と考えています。丸暗記に留まらない、広さと深さが望まれると思います。

3. そして、説明能力

AIへの指示やアプリ作成は要は部下に対する指示能力のお話です。次に求められるのは、その**成果に対する評価と、それを上司・お客様に説明し納得頂く能力**です。

4. 最後に、企画/開拓能力

AIを活用して成果を出せるなら、それを活用した新たな企画・提案や開拓が可能になるかもしれません。根底に好奇心があるとユニークになりやすいです。

5. それらも全部AIがやるかも？

可能性はあります。確率も結構高いかなとも思ってます

しかし現実がどうなるかは分かりません。分かった時に、**無駄だった可能性も確かにありますが、「あの時やっておけばよかった」の可能性もあります**。今やるべき事は試行錯誤です。肌感を掴むことが重要です。

知見としてのまとめは以上です。

これを逆に辿ることで、組織論ベースのノウハウから
AI活用の新たな方策が見出せたりするかも知ですが、
その辺はこれから模索です。

§ 6. 付録

ツール類の紹介

今興味もってるツールを適当に紹介

ワークフロー開発

AIを組織的な仕組み的に構築するお話をしました。それを実装するのに使えるツールを紹介しておきます。

- [Dify](#): ノーコードでワークフローを構築できます
 - [【初心者向け】Dify 詳細解説マニュアル](#)
- [n8n公式](#): これもノーコード最近よく見る
 - [【初心者向け】n8n入門 | ノーコードで始める業務自動化の基本実践ステップを徹底解説！](#)
- [Langflow](#): これもノーコード。知名度はDifyかな
 - [LangChainのGUI版、LangFlowを試してみた](#)
- [LangGraph](#): 要コーディング。老舗
 - [【初心者向け】LangGraphの紹介と基本的な使い方](#)
 - [LangGraphの基本的な使い方](#)
- [Mastra](#): TypeScriptベース

その他

- [LangChain](#): 汎用アプリ開発。LangGraphのベース
 - [そろそろ知っておかないとヤバイ？ 話題のLangChainを30分だけ触って理解しよう！](#)
 - [LangChain\(ラングチェーン\)で何ができる？機能・使い方から活用事例、商用利用の可否について紹介](#)
- [Langfuse](#): LLMの追跡・分析ツール
- [use cases@Claude](#): 業務ごとの活用例
- [Claude Code Templates](#): AIエージェントの例

③ 使いたいツールあればご意見ください
いずれもちょこっと触ってみたレベルなので
次何するか悩み中です。

思考軸を据える理由

大枠

新しい技術や概念に触れるときに、自分の中で消化(理解)するための小手先テクニックです。

具体的には、用語や考えを分類したり、お互いの関係を「この思考軸なら～」で位置づけたりすることで言語化するのに便利になると思います。

そのあと、更に新しい情報が入ってきた場合に、その情報の新規性や優先度を測るのにも使ったりします。

あくまで理解のための補助ですので、正しい訳でも、何かしらの本質でもなければ、そこに意味もありません。更に明日には他の軸に変えたりの可能性もあります。ただ、便利なテク、というお話です。

例)ハルシネーション

ソクラテス観点だと「未知の未知(知らないことに気づいてない)」です。

[OpenAI社がレポート](#)出していますが、「不確実なら棄権する」という選択肢を設ける事によりエラー率が低減しています。

個人的にはこの解決策は「既知の未知(知らないことに気づいてる)」に持ち込んだと解釈しています。

例)RAGとMPCとワークフロー

上記の3つとも、技術観点では「LLMのテキスト出力をチェックして外部機能を実行する」で原理は共通です。

しかし、機能観点では単にエビデンスの取得なのか、スキルの拡張なのかで異なり、AI的直感の観点では組織化によるサポートという考えに至ります。

本資料:<https://github.com/nab391/tech-study-notes/tree/main/AI>
アンケート:[アンケート@Googleフォーム](#)

おしまい

以上で終了です
AI活用はとっかかりのハードルが高いですが、
冒頭にお話した通り、古参メンバーが居ないだけでもコスパはよいので
今回のレクをきっかけに手に取って貰えたら幸いです。

fin